

Description of Lab Work nº 1	
Course	Real-Time systems / <i>Sistemas de Tempo Real</i>
Year	2025/2026
Aim	Study concepts of Real-Time Systems and develop a project with a real-time kernel
Classes	5+1 classes x 3 hours + 15 extra hours (outside)
Delivery Date	27/10/2023
Concrete Objectives: - Interaction with physical systems (Interface boards, sensors, actuators, ...) - Interface board (Data acquisition) - Sensors - Actuators - Interface functions (in C++) - Apply the base real-time system's concepts, namely: - Task / Process: concurrency, - State, event, actions (triggered by events) - Synchronization and communication between tasks - Accessing shared resources / exclusive access resources - Using Real-Time kernels - freeRTOS - C/C++ Language - Solve the problem described in Annex 1.	

Annex 1 – Description of Work

This Lab-work aims at training the real-time concepts with a real-time kernel. The work comprises the development of a program that allows the control of a manufacturing line that must route different types of **Construction Bricks** to comply with some construction's requirements, using real-time approaches.

The tools necessary to develop the work comprise a real-time kernel (freeRTOS), the Visual Studio 2019 C/C++ compiler and the hardware kit that represents the Splitter Conveyor, illustrated in Figure 1 and Figure 2. To enhance students' performance other tools are available for developing this work, such as a Splitter Conveyor simulator, and a set of low-level functions for software/hardware interaction, provided in the lab slides.

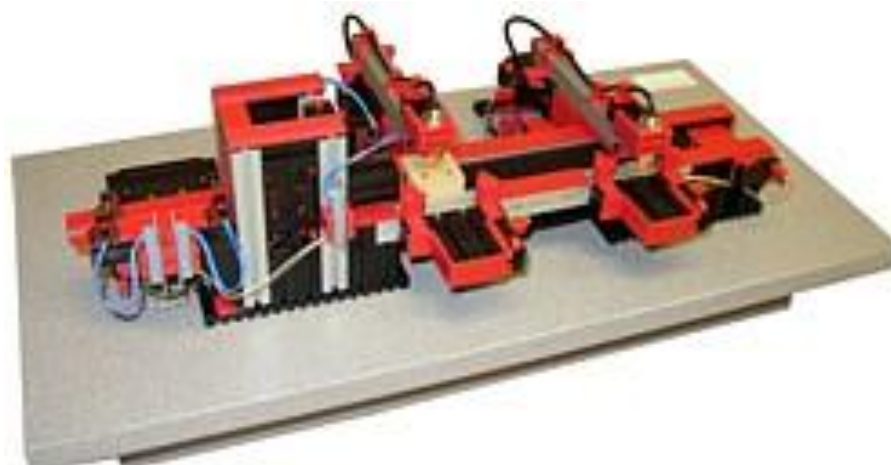


Figure 1– Kit which models the manufacturing assembly line.

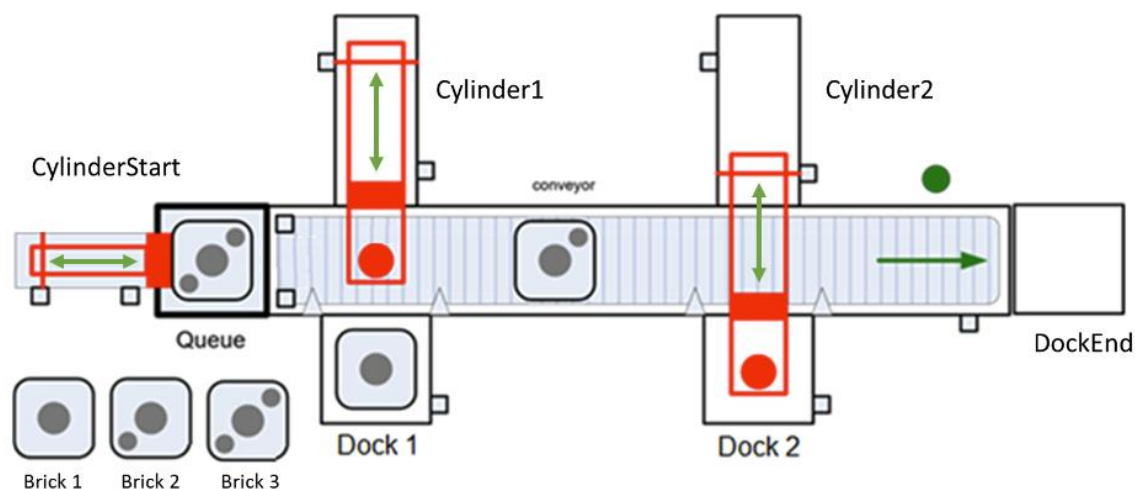


Figure 2 – Visual representation of the distribution line

The interaction between the real-time program and the Splitter Conveyor is performed through an input / output digital data acquisition board, specifically, the NI USB 6509, as shown in Figure 3.



Fig. 3 – the NI USB 6509 data acquisition board

The manufacturing line has got three distributions docks: Dock 1, Dock 2 and Dock End, which should work in the following way:

→ **NORMAL MODE:**

- The human operator must press key = 's' to start the system normal behaviour, and 'f' to finish the work.
- **Dock 1** receives Bricks of type 1.
- **Dock 2** receives Bricks of type 2.
- **Dock End** receives Bricks of type 3.
- The human operator must press key = 'p' to push a new brick into the conveyor belt.

→ **Operator override** (P1.4 for Dock 1, P1.3 for Dock 2)

- Each button press queues one **override token** for that dock. For each token, the next incoming brick (regardless of type) is forced to that dock.
- Multiple presses queue multiple tokens, e.g., two presses on P1.4 can yield a delivery pattern to Dock 1 like ... 1 1 3 3 1 1 ... (because two bricks were forced there).

→ **Batch/sequence rule & Lamp**

- Each dock maintains a batch counter of correctly-typed bricks delivered to that dock: Dock 1 counts only type 1 delivered to Dock 1; and Dock 2 counts only type 2 delivered to Dock 2.
- When a dock's counter reaches 3, Lamp at P2.7 flashes once, and that dock's counter resets to 0.
- Forced (override) bricks of the wrong type do not increment the counter.
- Any type 3 or any brick routed to **Dock End** triggers Lamp at P2.7 "many times" (3 cycles).

→ **Emergency**

- Press Dock End button → enter Emergency: stop all motion immediately; ignore/hold new actuations; keep internal queues/counters.

Secção de Robótica e Manufatura Integrada

- While in Emergency: Lamp at P2.7 flashes ON 500 ms / OFF 500 ms continuously (this has priority over other flashes).
- Press Dock End button again → resume exactly where paused (queued overrides and batch counters are preserved).

→ **Statistics** (must be available on-screen)

- Totals since start by brick type: count of type 1, type 2, type 3 entering the system.
- Per-dock deliveries by type (e.g., how many type 1, type 2, type 3 actually went to Dock 1, Dock 2, Dock End).
- Number of completed sequences.
- Override uses per dock (how many tokens consumed).
- Current mode (Normal/Emergency).

In terms of operationality, each brick is taken from the entry queue, using cylinder start, and placed into the conveyor. There are two sensors at the beginning of the conveyor, which can identify the brick type. For instance, a brick of type 3 contains two magnetic marks at their corners, in a diagonal way (see figure 3). Whenever these marks pass under the mentioned sensors, the corresponding brick can be identified.

However, there is a technological constraint on identifying each brick, because when cylinder start completes its onward movement to deliver a brick, the incoming brick is already half inside the conveyor. Therefore, using a non-concurrent approach, in which the brick is sequentially delivered and then identified, will fail because one of the marks already passed under the sensors, leading to misidentifying the brick. As such, it is recommended that the mentioned identification routine finishes identifying only after each brick arrives at the sensor in Cylinder 1 at Dock 1, in which the central mark is identified. Furthermore, a brick of type an only contains the center mark and cannot be spotted by the identification sensors.

Secção de Robótica e Manufatura Integrada

Finally, keep in mind that the system has active high and active low sensors. Active low sensors provide logical value false/0 when active, and true/1 otherwise.

The functional **requirements** to develop the **Splitter Lego Conveyor** are listed in the following table. They are split into basic or low-level requirements, for direct interaction with sensors and actuators, and high-level requirements, for storage management.

Table 1 – Needed requirements.

Req.	Description
Low-Level Requirements:	
FR1.1	Move and stop each cylinder
FR1.2	Move and stop moving the conveyor
FR1.3	Read Brick identification sensors
FR1.4	Detect the button/switches states
FR1.5	Turn Lamp on and off
FR1.6	Keyboard or UI interaction
High-Level requirements (operation):	
FR2	Using the keyboard, develop robust (semi-automatic) calibration . Using keys, the user defines the direction of the movements. During a movement, the actuator must stop when it reaches a position sensor (Cylinders)
FR3	Insert a brick into the system, which gets a brick from the entry queue and puts it in the conveyor
FR4	By pressing “s” the system Starts its working cycle. Pressing “f”, the system Finishes its work.
FR5	Deliver Brick1 for Dock 1 , Brick2 for Dock 2 , and Brick3 to DockEnd
FR6	Detection of buttons P1.4 and P1.3 pressed by human operator, at dock 1 and dock 2 to force specific bricks to the corresponding docks
FR7	Flash lamp at P2.7 once when delivering sequence at Dock 1 or Dock 2
FR8	Flash lamp at P2.7 (3 times) when routing a brick to Dock End
FR9	Handle Emergency STOP/RESUME operation with corresponding LED signalization

Secção de Robótica e Manufatura Integrada

High-Level requirements (information):	
FR10	Collect statistics of the total number of bricks that were delivered at each dock. Show these statistics on the console screen whenever requested.
Non-Functional requirements:	
NF1	While operating, the system can accumulate requests
NF2	While operating, the system can provide information (e.g., number of bricks delivered at dock 1)
NF3	The system works properly (e.g. (i) when placing a new brick on the conveyor, it must be moving; (ii) delivers corresponding bricks to docks according to the description of the system)
NF4	The system always moves within limits (cylinders)

Departamento de Engenharia Electrotécnica e de Computadores

Secção de Robótica e Manufatura Integrada

Classes plan for the Lab Work 1

Class 0:

SW Installation.

Class 1:

Interaction with “physical” system.

Class 2:

Interaction between a program and physical systems, through sensors, actuators, and a data acquisition board; Development of the functions for the interaction with the Splitter Conveyor.

Class 3:

Development of real-time behaviour; tasks and semaphores; development of the low-level requirements.

Class 4:

Communication mechanisms (mailboxes); development of the high-level requirements to control the Splitter Conveyor.

Class 5:

Conclusion of LabWork1.

Professors:

- | | |
|---|----------------------------------|
| – Filipa Ferrada, fam@fct.unl.pt | (theoretical/laboratory classes) |
| – André Rocha, ad.rocha@fct.unl.pt | (theoretical/laboratory classes) |
| – Ana Inês Oliveira, aio@fc.unl.pt | (laboratory classes) |
| – Ricardo Peres, ra.peres@fct.unl.pt | (Laboratory classes) |

Lab Work 1 Delivery:

Students must submit via Moodle **(and only via Moodle)**:

- ➔ Upload a zip/rar archive with your Visual Studio Project.
- ➔ Upload a demo vídeo (**aprox. 5 minutes, videos with more than 7 minutes will not be considered**) presenting all implemented requirements/functionality. The video **must be** accompanied with a narration of what is being presented.

Departamento de Engenharia Electrotécnica e de Computadores

Secção de Robótica e Manufatura Integrada

Evaluation criteria

Items			Quotation/ Penalty
Functionality	FR1	Low-Level Requirements	1.0
	FR2	Using the keyboard, develop robust (semi-automatic) calibration. Using keys, the user defines the direction of the movements. During a movement, the actuator must stop when it reaches a position sensor (Cylinders)	0.5
	FR3	Insert a brick into the system, which gets a brick from the entry queue and puts it in the conveyor	1.0
	FR4	By pressing "s" the system Starts its working cycle. Pressing "f", the system Finishes its work.	1.0
	FR5	Deliver Brick1 for Dock 1, Brick2 for Dock 2, and Brick3 to DockEnd	1.5
	FR6	Detection of buttons P1.4 and P1.3 pressed by human operator, at dock 1 and dock 2 to force specific bricks to the corresponding docks	2.0
	FR7	Flash lamp at P2.7 once when delivering sequence at Dock 1 or Dock 2	1.0
	FR8	Flash lamp at P2.7 (3 times) when routing a brick to Dock End	1.0
	FR9	Handle Emergency STOP/RESUME operation with corresponding LED signalization	2.0
	FR10	Collect statistics of the total number of bricks that were delivered at each dock. Show these statistics on the console screen whenever requested.	1.5
	NF1	While operating, the system can accumulate requests	1.5
	NF2	While operating, the system can provide information (e.g., number of bricks delivered at dock 1)	2.0
	NF3	The system works properly (e.g. (i) when placing a new brick on the conveyor, it must be moving; (ii) delivers corresponding bricks to docks according to the description of the system)	1.0
	NF4	The system always moves within limits (cylinders)	1.0
Video			2
TOTAL			20
No delivery of the VS22 project <u>or</u> the video (* MANDATORY)			-20